SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Course	Digital Signal Processing Lab	**Experiment No.**	07
Name	Dewashish Lambore	**PRN**	_____
Language	Python 3.10	**Date**	____ / ____ / 2026

Difference-Equation Based Smoothing

Smoothing the US dollar to Indian rupee exchange rate

Abstract: Seven years of the daily USD/INR exchange rate (1,748 trading days, ₹70.02 to ₹89.84) were smoothed with five filters, all written and solved as linear constant-coefficient difference equations: the 20-day moving average in non-recursive and recursive form, the 20-day exponential moving average (EMA), a cascaded double EMA and Holt's trend-corrected smoother. The iterative solver matched a reference implementation to 1e-12. With the finance convention $\alpha = 2/(N + 1)$, the EMA and the moving average have the same lag (9.5 days) and the same noise gain (0.050), but the EMA needs 2 multiplications per sample instead of 20. Across a family of lags, both follow the same law, noise gain $= 1/(2L + 1)$. The initial condition $y[-1] = 0$ caused a transient of 47 days to fall below 1 %, exactly as the pole at $z = 0.905$ predicts; on the 2022 fall of the rupee the trend-corrected second-order smoother cut the lag from 7.6 to 0.2 days at a modest cost in noise.

Keywords: *difference equation, recursive filter, exponential moving average, moving average, poles, stability, initial conditions, lag, MACD, exchange rate, Python*

1. Aim and Objectives

Aim: to design, solve and analyse smoothing filters described by difference equations, and to apply them to a real noisy signal in Python.

- Solve a general difference equation by iteration, including initial conditions, and split the response into its zero-input and zero-state parts.
- Compare recursive (IIR) and non-recursive (FIR) smoothers by impulse response, poles, stability, lag, noise reduction and computational cost.
- Apply the smoothers to the USD/INR rate and quantify the lag-versus-smoothness trade-off.

2. Real-World Application

Importers, exporters, banks and the Reserve Bank of India watch the rupee every day, and the day-to-day wiggles (standard deviation 0.30 %) hide the underlying trend. Every trading terminal therefore draws moving averages, and the most common one, the exponential moving average, is nothing but a first-order recursive difference equation. The same few lines of code smooth data everywhere (Table 1); choosing them is always a trade-off between how much noise is removed and how late the smoothed signal reacts. Note: this experiment studies signal processing properties only; it does not evaluate trading strategies.

Table 1. Where difference-equation smoothers are used

Field	Signal	Difference-equation smoother used
Finance, forex desks	exchange rates, prices	EMA, MACD (difference of two EMAs), Holt trend
Sensors and IoT	temperature, battery voltage	first-order IIR low-pass in firmware
Navigation	GPS position, speed	alpha-beta filter (Holt) and Kalman filter
Web and operations	page views, server latency	EWMA baselines for anomaly alarms
Audio	level meters, envelopes	one-pole attack / release smoothing

3. Theory

A linear time-invariant system with a finite number of delays is described by a linear constant-coefficient **difference equation**. With $a_0 = 1$ it can be solved forward in time, one sample at a time, from the input and the previous outputs (the initial conditions):

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k] - \sum_{k=1}^{N} a_k\, y[n-k] \tag{1}$$

If any $a_k \neq 0$ ($k \geq 1$) the filter is **recursive** (IIR); otherwise it is non-recursive (FIR). Its transfer function is a ratio of polynomials whose roots are the zeros and poles; the filter is stable if every pole lies inside the unit circle:

$$H(z) = \frac{\sum_k b_k z^{-k}}{1+\sum_k a_k z^{-k}}, \qquad \text{stable} \Leftrightarrow |p_i| < 1 \tag{2}$$

The **exponential moving average** has one pole at $z = 1-\alpha$ and a geometric impulse response; finance chooses α from a nominal window N:

$$y[n] = \alpha\, x[n] + (1-\alpha)\, y[n-1], \qquad h[n] = \alpha(1-\alpha)^n u[n], \qquad \alpha = \frac{2}{N+1} \tag{3}$$

The N-point moving average is FIR, but it can also be written recursively with only one update per sample; this creates a pole at $z = 1$ that is cancelled by a zero:

$$y[n] = y[n-1] + \frac{x[n]-x[n-N]}{N} \tag{4}$$

The complete response of a recursive filter is the sum of the **zero-input** response (due only to the initial condition) and the **zero-state** response (due only to the input). For the EMA:

$$y[n] = \underbrace{(1-\alpha)^{n+1}\, y[-1]}_{y_{zi}[n]} + \underbrace{\sum_{k=0}^{n} \alpha(1-\alpha)^k\, x[n-k]}_{y_{zs}[n]} \tag{5}$$

Two numbers summarise a smoother: the **noise gain** (output variance for unit-variance white noise) and the **lag** (low-frequency group delay):

$$G = \sum_n h^2[n], \qquad L = \frac{\sum_n n\, h[n]}{\sum_n h[n]}; \qquad \text{EMA: } G = \frac{\alpha}{2-\alpha}, L = \frac{1-\alpha}{\alpha} \tag{6}$$

For the moving average $G = 1/N$ and $L = (N-1)/2$. Eliminating N and α shows that both obey the same law $G = 1/(2L+1)$; with $\alpha = 2/(N+1)$ the EMA has exactly the lag of the N-day moving average. **Holt's smoother** adds a second state for the trend, giving a second-order equation $l[n] + a_1 l[n-1] + a_2 l[n-2] = \alpha x[n] - \alpha(1-\beta)x[n-1]$ with $a_1 = \alpha + \alpha\beta - 2$, $a_2 = 1-\alpha$, whose lag on a straight-line trend is zero.

Note. A filter with a pole on the unit circle, such as the recursive moving average, is only marginally stable: it works because the zero cancels the pole exactly, but rounding errors are never forgotten. Long-running systems therefore recompute the sum from scratch periodically.

4. Dataset Description

Series **DEXINUS** (Indian rupees to one US dollar, noon buying rate in New York) is published by the Board of Governors of the Federal Reserve (H.10 release) and distributed free by FRED, the Federal Reserve Bank of St. Louis. The CSV for 2019 to 2025 is 33 kB and is cached locally. It has 1,826 weekday rows, of which 78 are holidays without a quotation; dropping them leaves 1,748 samples, so the index n counts **trading days**.

Table 2. Dataset attributes

Property	Value	Property	Value
Source	FRED series DEXINUS (Federal Reserve H.10)	Unit	₹ per US dollar
Period	02 Jan 2019 to 31 Dec 2025	Samples	1,748 trading days
Lowest	₹68.40 (10 Jul 2019)	Highest	₹91.03 (16 Dec 2025)
Daily change, std	0.296 %	Largest daily move	1.93 %
2022 move (zoom)	₹74.32 to ₹82.72	Licence	public domain (US government data)

Table 3. Dataset preview with the EMA output (initial condition y[−1] = x[0])

n	Date	x[n] (₹/US$)	Daily change (%)	EMA20 y[n]	x − y
0	02 Jan 2019	70.0200	n/a	70.0200	+0.0000
1	03 Jan 2019	70.1200	+0.143	70.0295	+0.0905
2	04 Jan 2019	69.6300	-0.699	69.9915	-0.3615
3	07 Jan 2019	69.7800	+0.215	69.9713	-0.1913
4	08 Jan 2019	70.1100	+0.473	69.9845	+0.1255
5	09 Jan 2019	70.4200	+0.442	70.0260	+0.3940

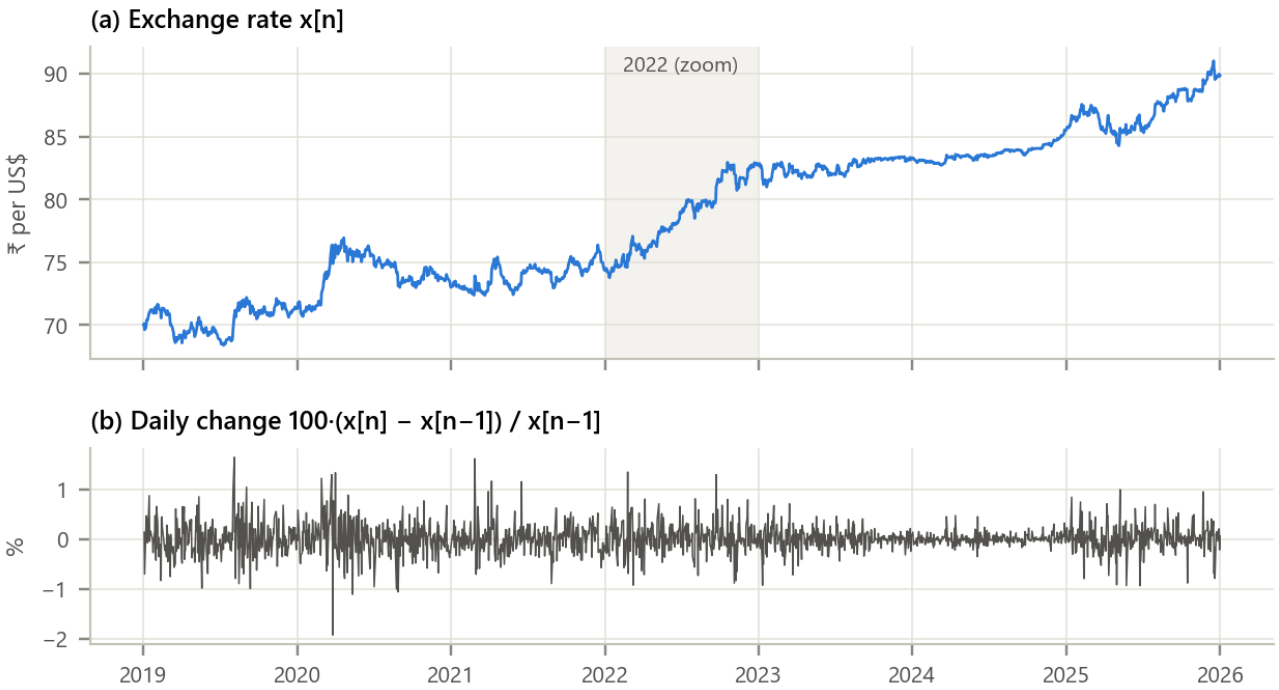


Figure 1. The input signal. (a) Daily exchange rate with the 2022 zoom window shaded; (b) daily percentage change.

Inference. The rupee weakened from about ₹70 to ₹90 per dollar, with a sharp fall in 2022 (₹74.3 to ₹82.7) and long flat stretches in 2023 when the central bank stabilised the rate. The daily changes look like noise around zero (std 0.30 %, largest 1.93 % in March 2020). A good smoother should keep the slow trend of (a) and remove the fast fluctuations of (b).

5. Methodology and Implementation

Every filter is defined only by its (b, a) coefficients and passed to one iterative solver of Eq. 1, which was cross-checked against SciPy's `lfilter`. Filters start from y[−1] = x[0] unless stated. Parameters: N = 20 (α = 0.0952), Holt β = 0.05, MACD 12/26/9. Lag is measured on the 2022 trend as mean tracking error divided by the trend slope.

Table 4. Software modules

Module	Responsibility
`config.py`	FRED series, period, filter parameters, zoom window
`data_loader.py`	Download once, cache, drop holidays, attach trading-day index n
`smoothing.py`	Iterative difference-equation solver, five smoothers, poles, H(e^jω), gain and lag
`analysis.py`	Filter table, trade-off study, initial-condition study, MACD
`plots.py` / `gui.py`	Report figures / interactive smoother designer
`test_smoothing.py`	11 unit tests (lfilter agreement, zi + zs, formulas, Holt, instability): all pass

Core of smoothing.py

```python
def difference_equation(b, a, x, y_init=None, x_init=None):
    """y[n] = ( sum b_k x[n-k] - sum_{k>=1} a_k y[n-k] ) / a_0, solved sample by sample"""
    M, N = len(b) - 1, len(a) - 1
    xp = np.zeros(M) if x_init is None else np.asarray(x_init, float)   # x[-1] ... x[-M]
    yp = np.zeros(N) if y_init is None else np.asarray(y_init, float)   # y[-1] ... y[-N]
    y = np.zeros(len(x))
    for n in range(len(x)):
        y[n] = (np.dot(b, np.r_[x[n], xp]) - np.dot(a[1:], yp)) / a[0]
        xp = np.r_[x[n], xp[:-1]] if M else xp                          # shift the delay lines
        yp = np.r_[y[n], yp[:-1]] if N else yp
    return y

ema_coeffs = lambda alpha: ([alpha], [1.0, -(1.0 - alpha)])          # y = a x + (1 - a) y[n-1]
ma_coeffs = lambda N: (np.full(N, 1.0 / N), [1.0])                    # non-recursive average

```

GUI. `gui.py` (Figure 2) offers the five smoothers or any typed b and a coefficients, the window N, Holt's β, the initial condition and the year to display. It shows the smoothed rate, the impulse response, the magnitude response and the pole-zero plot, and reports stability, lag, noise gain and the cost in multiplications per sample. Run `python run_experiment.py` and `python gui.py`.

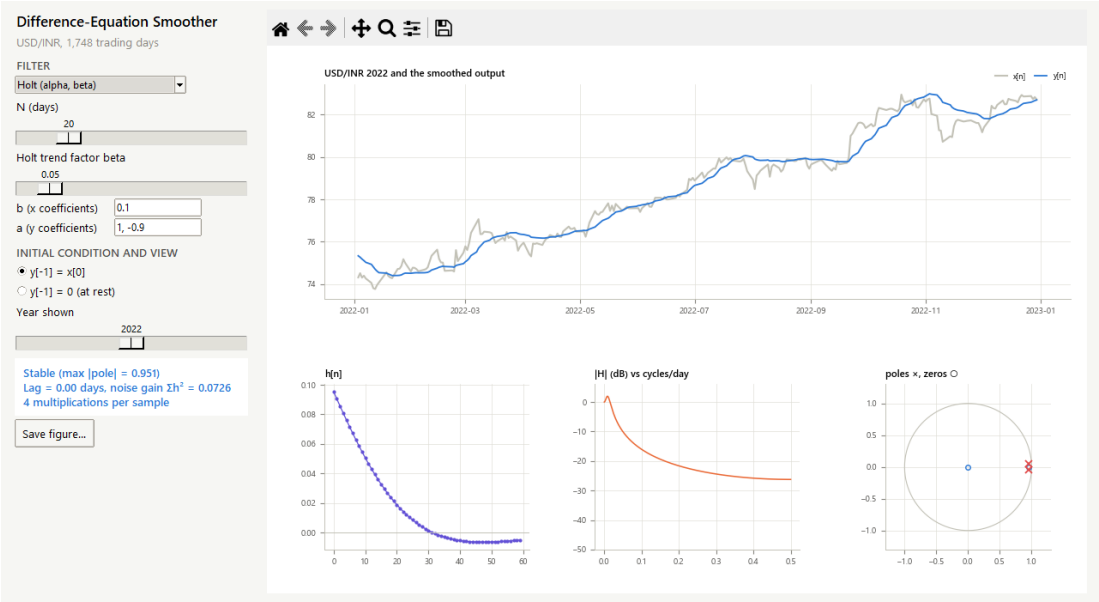


Figure 2. The difference-equation smoother designer showing Holt's trend-corrected filter. Two complex poles near z = 0.95, zero lag and a small low-frequency overshoot in |H|.

6. Calculations

By hand. EMA with α = 0.5 on x = {80, 82, 81, 85}, first with y[−1] = 80 and then starting at rest (y[−1] = 0); y[n] = 0.5 x[n] + 0.5 y[n−1]:

Table 5. Difference equations solved by iteration

n	x[n]	y[n], y[−1] = 80	y[n], y[−1] = 0	Difference = y_zi[n] = $0.5^{n+1}\cdot 80$	3-point MA
0	80	0.5·80 + 0.5·80 = **80**	40	40	n/a
1	82	0.5·82 + 0.5·80 = **81**	61	20	n/a
2	81	0.5·81 + 0.5·81 = **81**	71	10	(80 + 82 + 81)/3 = 81.00
3	85	0.5·85 + 0.5·81 = **83**	78	5	(82 + 81 + 85)/3 = 82.67

The difference between the two columns is exactly the zero-input response, as Eq. 5 states. For the 20-day EMA used on the data, $\alpha = 2/21 = 0.0952$, so the pole is at $z = 0.9048$; the initial condition decays below 1 % after $n = \ln(0.01)/\ln(0.9048) = 46.0$, i.e. **47 trading days**. Noise gain $\alpha/(2-\alpha) = 0.0500$ and lag $(1-\alpha)/\alpha = 9.50$ days, identical to the 20-day moving average (1/20 = 0.05 and 19/2 = 9.5). These values agree with the program (checked by `test_smoothing.py`).

7. Results and Discussion

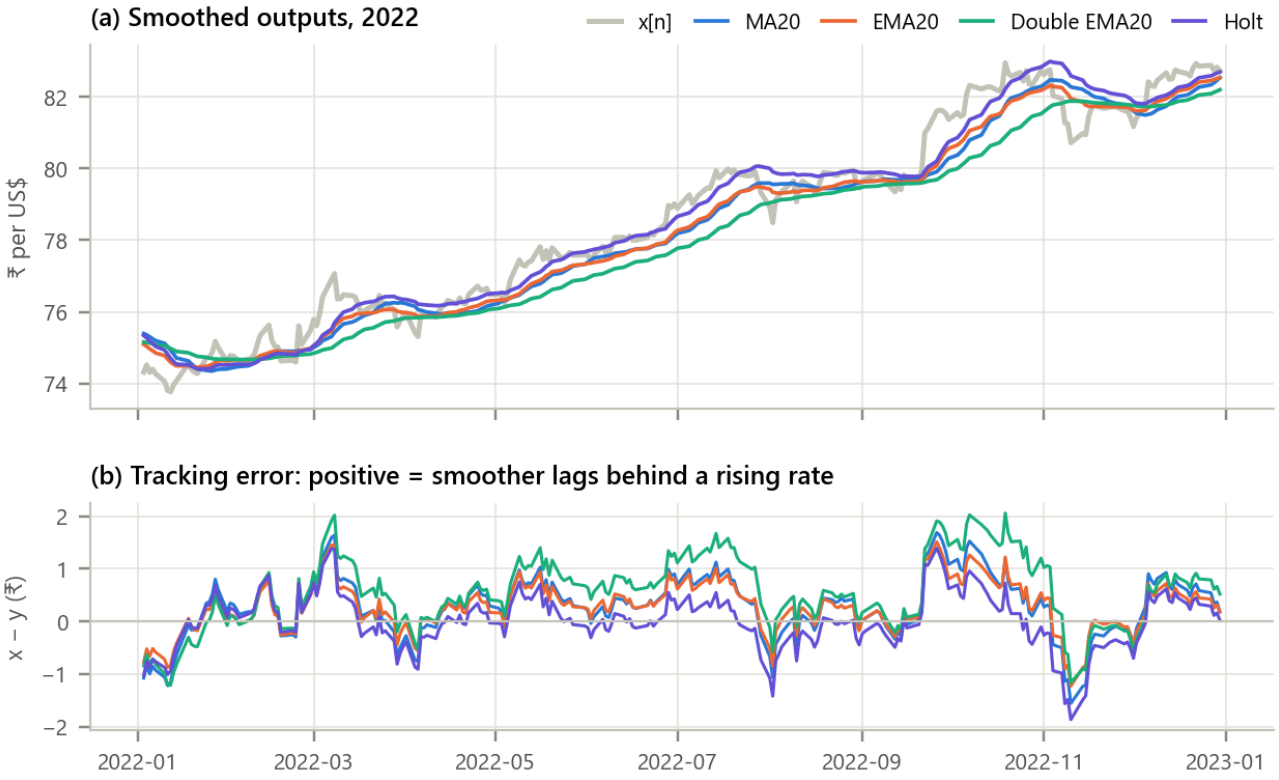


Figure 3. The smoothers applied to the 2022 fall of the rupee. (a) Input and smoothed outputs; (b) tracking error x − y.

Inference.

- All four smoothers remove the day-to-day wiggles; the double EMA is the smoothest and Holt's filter the most responsive.
- While the rupee falls steadily, every first-order smoother runs **behind** the rate (positive error). Measured on the trend, the moving average lags 7.7 days, the EMA 7.6, the double EMA 14.9 and Holt only 0.2 days. They are somewhat below the theoretical 9.5, 9.5, 19.0 and 0 days because 2022 was not a perfect straight line.
- Holt's error swings around zero instead: by estimating the trend it removes the lag but overshoots when the trend stops, which is why its noise gain is higher (0.073).

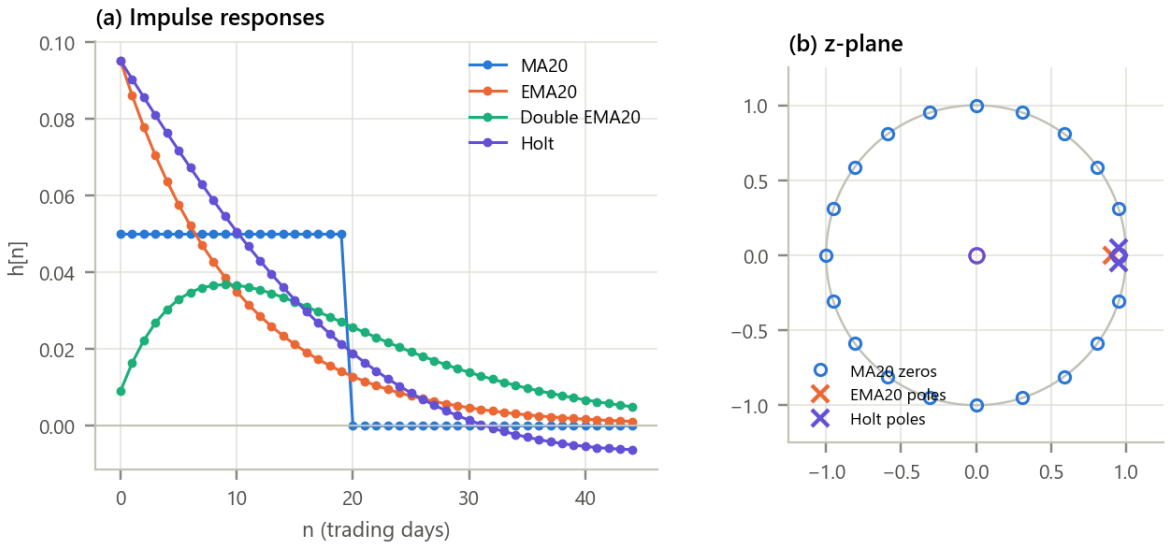


Figure 4. Impulse responses and pole-zero locations. (a) h[n] obtained by feeding δ[n] to the difference equation; (b) poles (×) and zeros (○).

Inference. The moving average has a rectangular, finite h[n] and no poles except at the origin; its 19 zeros sit on the unit circle and produce the nulls of its frequency response. The EMA's h[n] decays geometrically from α = 0.095 with a single pole at z = 0.9048. The double EMA has a double pole at the same place and a delayed, bell-shaped h[n]. Holt's filter has two complex poles of radius 0.951 and an h[n] that dips below zero, the negative weights that cancel the lag. All poles are inside the unit circle, so all four are stable.

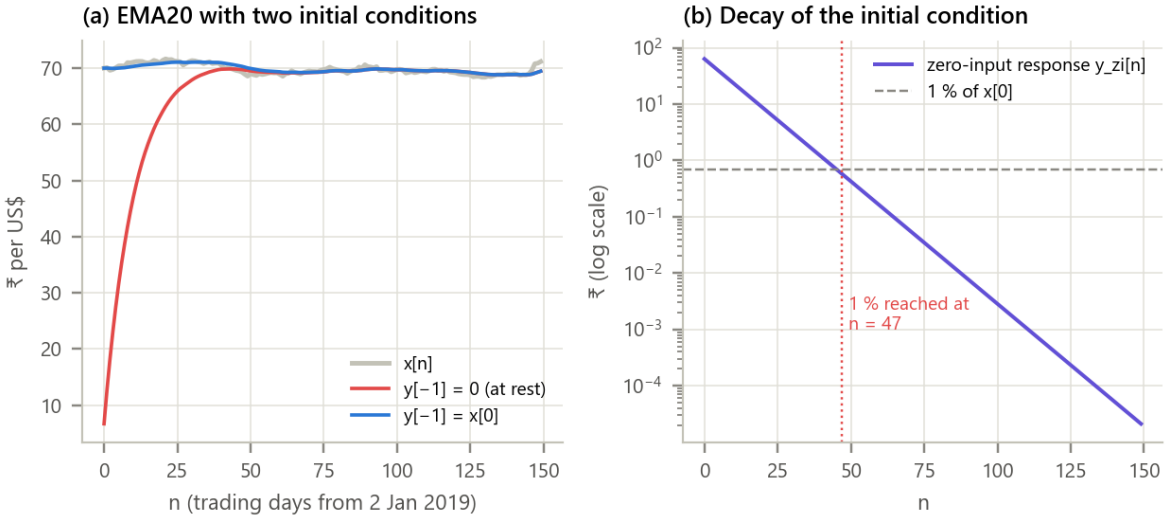


Figure 5. Effect of the initial condition on the EMA. (a) Starting at rest versus starting at x[0]; (b) the zero-input response on a logarithmic scale.

Inference. Starting at rest, the first output is only 6.67 instead of 70.02, an error of ₹63.35. The difference is exactly the zero-input response (decomposition error 3e-14), a straight line on the log scale because it shrinks by the pole value 0.905 every day. It falls below 1 % after 45 to 47 days, matching the theory, so the first two months of any EMA started at zero must be discarded.

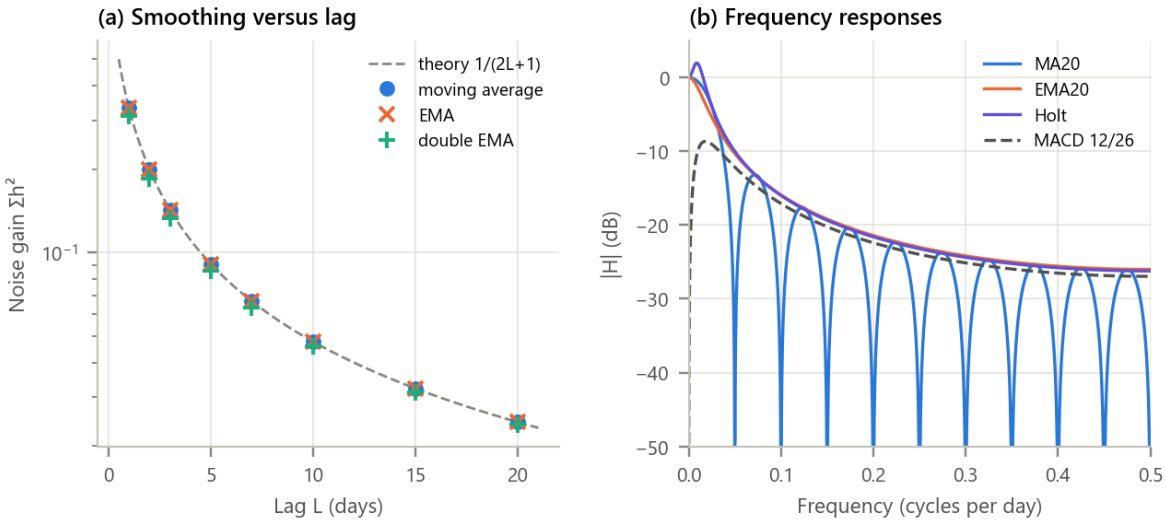


Figure 6. Smoothing versus lag, and the frequency domain. (a) Noise gain against lag for moving averages, EMAs and double EMAs; (b) |H(e^jω)| of the main filters and of MACD.

Inference.

- All three families lie on the same curve G = 1/(2L + 1): for white noise, **no linear smoother of this kind removes more noise for the same lag**; at lag 20 the double EMA is better by only 2 %. What differs is cost and shape.
- In frequency, the EMA rolls off smoothly while the moving average has deep nulls at multiples of 1/20 cycles per day and strong sidelobes. Holt's |H| rises slightly above 0 dB at very low frequency, the overshoot seen in Figure 3. MACD has zero gain at DC and peaks at a period of about 55 days: it is a band-pass filter that removes the long-term level.

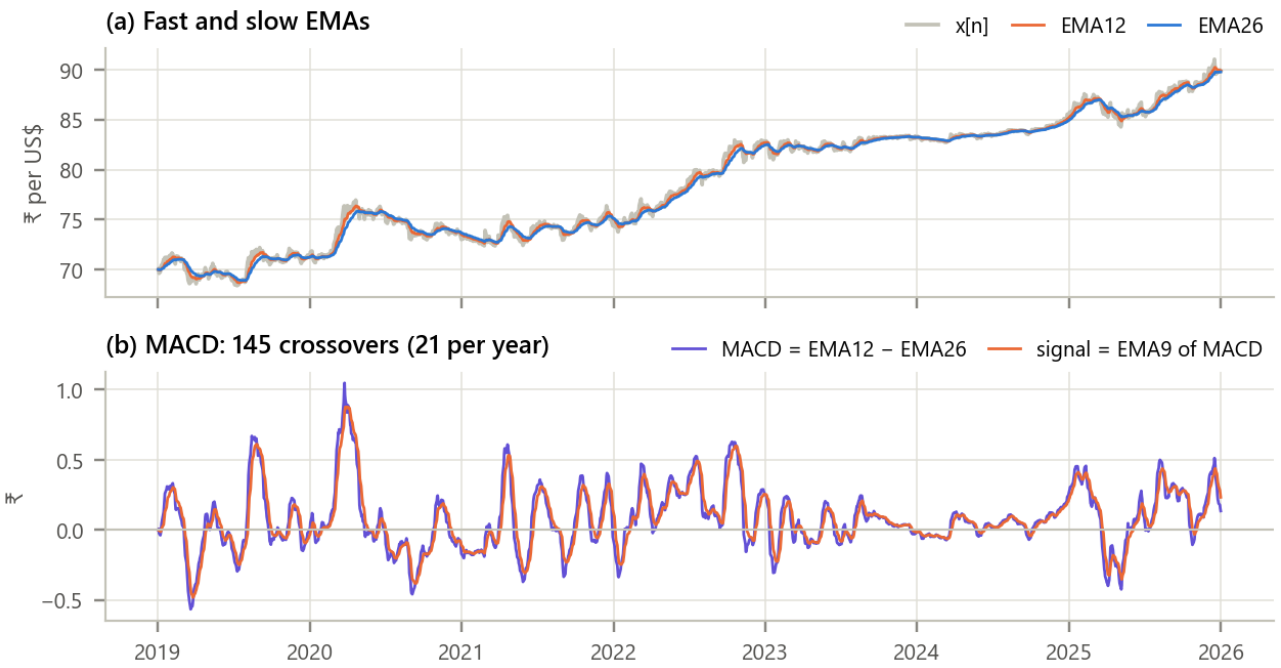


Figure 7. Application: the MACD trend indicator built from two EMAs. (a) Rate with EMA12 and EMA26; (b) MACD line and its 9-day EMA signal line.

Inference. MACD is simply the difference of two difference equations followed by a third. It oscillates around zero, positive while the rupee weakens (rate rising) and negative when it recovers. The MACD and signal lines crossed 145 times in seven years, about 21 times a year; each crossing is a change in the short-term trend as seen through filters with roughly 10 to 20 days of lag, which is why such signals always arrive after the turning point.

Table 6. Summary of the smoothers

Smoother	Order (M, N)	Max |pole|	Stable	Lag theory	Lag on 2022	Noise gain	Mult./sample
MA20 FIR	(19, 0)	0.000	yes	9.5	7.7	0.0500	20
MA20 recursive	(20, 1)	1.000	marginal	9.5	7.7	0.0500	3
EMA20	(0, 1)	0.905	yes	9.5	7.6	0.0500	2
Double EMA20	(0, 2)	0.905	yes	19.0	14.9	0.0251	3
Holt (level + trend)	(1, 2)	0.951	yes	0.0	0.2	0.0726	4

8. Conclusion

Five smoothers were expressed as difference equations, solved by direct iteration and verified against an independent implementation. The analysis showed that the familiar EMA is the cheapest of them (two multiplications per sample against 20 for the moving average) while matching the moving average's lag and noise reduction exactly when $\alpha = 2/(N + 1)$, and that every first-order or cascaded smoother obeys the same trade-off $G = 1/(2L + 1)$: more smoothing always costs more delay. Recursion brings its own concerns: the initial condition lingers for 47 days and a pole on the unit circle makes a filter only marginally stable. Breaking the trade-off requires a model of the signal: Holt's second-order equation, which estimates the trend, tracked the 2022 fall with almost no lag (0.2 days) at the price of overshoot, the same idea that underlies alpha-beta and Kalman filters in navigation.

9. Learning Outcomes

- Write smoothing filters as difference equations and solve them by iteration with initial conditions.
- Find poles, zeros, impulse and frequency responses, and judge stability from the pole locations.
- Separate the zero-input and zero-state responses and estimate transient length.
- Quantify the lag versus noise trade-off and choose recursive or non-recursive designs on cost and behaviour.

10. References

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010, ch. 2 and 6.

[2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing*, 4th ed. Pearson, 2007, ch. 2.4.

[3] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *Int. J. Forecasting*, vol. 20, no. 1, pp. 5-10, 2004 (reprint of 1957 report).

[4] Board of Governors of the Federal Reserve System, "H.10 Foreign Exchange Rates," via FRED, series DEXINUS.

[5] G. Appel, *Technical Analysis: Power Tools for Active Investors*, FT Press, 2005 (MACD).